理解数的表示系统（Understanding Number Representation Systems）

十进制系统（基数-10，Decimal System / Base-10）

- 我们最熟悉的数制，由数字 0 到 9 组成。
- 数字被表示为 10 的幂之和。
- 示例：

  十进制数 147.3 展开为：

  $1 \times 10^2 + 4 \times 10^1 + 7 \times 10^0 + 3 \times 10^{-1}$

N 进制系统（Base-N Systems）

- 任何数制都可以使用基数 N，其中 N 是该系统中不同数字（digit）的个数。
- 示例（三进制，Base-3）：

  121（三进制）= $1 \times 3^2 + 2 \times 3^1 + 1 \times 3^0$ = 16（十进制）

二进制数（基数-2，Binary Numbers / Base-2）

- 计算机的关键数制，只使用数字 0 和 1。
- 每一位数字都是 2 的幂的系数，也称为一个“比特”（bit）。

转换示例（Conversion Examples）

- 11（十进制）转换为二进制：

  11（十进制）= 1011（二进制）

- 37（十进制）转换为二进制：

  37（十进制）= 100101（二进制）

二进制运算（Binary Arithmetic）

二进制加法与乘法（Binary Addition and Multiplication）

- 加法示例：

  用二进制计算 37 + 17：

  100101 + 10001 = 110110，即十进制 54。

- 乘法示例：

  用二进制计算 37 × 17：

  100101 × 10001 = 1001110101，即十进制 629。

为什么二进制对计算机有用（Why Binary is Useful in Computers）

- 对二进制数的运算通过逻辑门（AND、OR、NOT）实现，计算机执行起来速度非常快。

局限性（Limitations）

- 二进制数以固定位长存储（例如 32 位系统），这限制了可表示数的范围。

浮点数（Floating Point Numbers）

什么是浮点数？（What are Floating Point Numbers?）

- 浮点数用于在计算机中表示实数，在使用固定位数（bit）的同时提供广泛的取值范围。
- 由于科学和工程计算需要高精度和广泛的取值范围，浮点数至关重要。

浮点数的组成部分（Components of Floating Point Numbers）：

- 符号（Sign，s）：决定数字是正数还是负数。
- 指数（Exponent，e）：2 的幂，表示数字的量级（scale）。
- 尾数（Fraction，f）：有效数字或尾数（mantissa），是介于 1 和 2 之间的值，表示数字的精度。

IEEE 754 双精度格式（IEEE 754 Double Precision Format）：

- Python 将浮点数映射为 IEEE 754 双精度格式，共使用 64 位：
    - 1 位用于符号，11 位用于指数，52 位用于尾数。
- 指数具有 1023 的偏置（bias），即用 e – 1023 对指数进行归一化。

计算与精度（Calculations and Precision）

浮点数的表示（Representation of a Floating Point Number）：

- 一个浮点数可以表示为：

```
n = (−1)^s × 2^(e−1023) × (1 + f)
```

示例（Example）：

- 对于二进制数：

```
1 10000000010 1000000000000000000000000000000000000000000000000000
```

- 符号 s = 1（负数）
- 指数 e = 1026（十进制为 3）
- 尾数 f = 1.5

精度注意事项（Precision Considerations）：

- 相邻两个浮点数之间的间隔（gap）随着数值大小的增大而增大。
- 函数 numpy.spacing(x) 可用于计算数字之间的间隔，该间隔随 x 增大而增大。

特殊情形与局限性（Special Cases and Limitations）

特殊的浮点数情形（Special Floating Point Cases）：

- 次正规数（Subnormal Numbers）：当指数为零时出现，可以表示非常小的数。
- 无穷（Infinity）与 NaN（非数，Not a Number）：当指数全为 1（即 2047）时出现。若尾数为零，则结果为无穷；否则结果为 NaN。

处理溢出与下溢（Handling Overflow and Underflow）：

- 溢出（Overflow）：当结果超过可表示的最大数时出现，Python 将其处理为 inf。
- 下溢（Underflow）：当结果小于可表示的最小的次正规数时出现，Python 将其处理为 0。

理解数值计算中的舍入误差（Understanding Round-Off Errors in Numerical Computations）

什么是舍入误差？（What are Round-Off Errors?）

- 舍入误差（round-off error）是由于数字存储到计算机内存时的近似而产生的。
- 这些误差之所以出现，是因为计算机用二进制表示浮点数，而二进制并不能总是精确地表示十进制数。
- 关键概念：数字的真实值与其在系统中的近似值之间的差，称为舍入误差。

表示误差（Representation Error）：

- 这是舍入误差的常见原因。
- 示例：π（3.14159…）或 1./3.（0.33333…）无法被精确表示，导致计算中出现微小但会累积的误差。
- 多次舍入（Rounding Multiple Times）：连续舍入会加剧误差。例如：
    - 将 4.845 舍入到两位小数得 4.85，再舍入到一位小数得 4.9，所产生的累积误差比直接舍入到一位小数（得 4.8）更大。

Python 中舍入误差的示例（Examples of Round-Off Errors in Python）

浮点运算问题（Floating Point Arithmetic Issues）

- Python 中的运算展示了浮点数如何累积舍入误差。
- 示例：
    - Python 中 4.9 – 4.845 的结果是 0.055000000000000604，而不是精确的 0.055。
    - 由于舍入近似，0.1 + 0.2 + 0.3 != 0.6。

舍入误差的累积（Accumulation of Round-off Error）：

- 误差在重复计算中不断累积。
- 示例：多次加减 1/3：
    - 如果只做一次：1 + 1/3 – 1/3 的结果为 1
    - 重复 100 次后再求值：结果变为 1.0000000000000002
    - 随着重复次数的增加，误差成比例地增大。